Pixel Sim — Developer Guide

---

1. What is `__pycache__` doing here?

Normally `__pycache__` just stores Python's bytecode (`.pyc` files) so the interpreter doesn't re-parse your source on every launch — a minor speed-up.

In this project it does something much bigger.

Numba's `@nb.njit(cache=True)` compiles your Python functions all the way down to native machine code (x86-64 / ARM assembly) the first time they run. That compilation takes 2–10 seconds. With `cache=True` Numba serialises the compiled native code into the `__pycache__` folder as `.nbi` + `.nbc` file pairs. On every subsequent launch it loads the pre-compiled binary directly — startup is instant and the compiled code is just as fast.

```
__pycache__/
  sim_fixed.cpython-311.pyc          ← normal Python bytecode (tiny)
  simulate_step.simulate_step-....nbi  ← Numba metadata
  simulate_step.simulate_step-....nbc  ← Numba compiled native binary
  _react._react-....nbi
  _react._react-....nbc
  ... (one pair per @nb.njit function)
```

When does Numba invalidate the cache automatically?

Numba re-compiles (and overwrites the cache entry) when:

- The **source code** of the decorated function changes
- The **argument types** change (e.g. you pass float64 instead of float32)
- The **Numba version** changes

It does **not** invalidate when:

- You rename the file
- You move the file to a different folder
- You leave old copies of `.nbi`/`.nbc` files from deleted functions lying around

That last point is where stale cache entries accumulate.

---

2. Does the cache make adding new elements harder?

No — the cache is completely transparent to you as a developer.

You write normal Python/Numba code, and the cache handles itself. The only thing to be aware of:

> If you edit a `@nb.njit` function, Numba detects the change via a hash of the function's bytecode and recompiles automatically on the next run. You don't need to delete anything manually.

The one time you *do* want to clear the cache is when you have leftover `.nbi`/`.nbc` files from functions you've deleted or renamed — they waste disk space but cause no correctness problems.

---

3. Step-by-step: adding a new element

We'll use **Oil** as the example — a flammable liquid that floats on water.

Step 1 — Define the colour

Add a colour constant near the top of the file with the other palette entries:

```python
OIL = (180, 140, 60)
```

Step 2 — Register the element

Add one line in the element registration block (order matters — IDs are positional, so always add at the **end**):

```python
Oil = Element("Oil", 0.85, 2, OIL, viscosity=0.02)
```

Parameter	Meaning
`"Oil"`	Name string (used for ID lookups)
`0.85`	Density — less than Water (1.0) so it floats on top
`2`	Move type: `0`=static, `1`=powder, `2`=liquid, `3`=gas, `4`=fire
`OIL`	RGB colour tuple
`viscosity`	Probability (0–1) of skipping movement each frame

Step 3 — Wire up the keyboard shortcut

In `on_key_event`, extend the mapping dict. The number you assign maps to the element's

position in the registration order (1-indexed):

```python
mapping = {
    keys.NUMBER_1: 1,   # Sand
    keys.NUMBER_2: 2,   # Water
    keys.NUMBER_3: 3,   # Stone
    keys.NUMBER_4: 4,   # Lava
    keys.NUMBER_5: 5,   # Obsidian
    keys.NUMBER_6: 6,   # Fire
    keys.NUMBER_7: 7,   # Steam
    keys.NUMBER_8: 8,   # Smoke
    keys.NUMBER_9: 9,   # Oil  ← add this
}
```

That's it for a simple element. Oil will now float on water, sink through air, and spread sideways — all handled automatically by the existing liquid physics and density system.

---

4. Step-by-step: adding a new reaction

Reactions are interactions between two specific cell types. We'll make **Oil + Fire → spread Fire** (oil burns).

Step 1 — Cache the new element's ID in `__init__`

At the bottom of the ID-caching block in `__init__`:

```python
ids = {data["name"]: eid + 1 for eid, data in ELEMENTS.items()}
self._steam_id = ids["Steam"]
self._water_id = ids["Water"]
self._lava_id  = ids["Lava"]
self._obs_id   = ids["Obsidian"]
self._oil_id   = ids["Oil"]    # ← add this
self._fire_id  = ids["Fire"]   # ← add this if not already present
```

Step 2 — Pass the new IDs into `simulate_step`

Add `oil_id` and `fire_id` to the function signature:

```python
```

```
# In _run_sim_step():
n = simulate_step(
    ...,
    self._steam_id, self._water_id, self._lava_id, self._obs_id,
    self._oil_id, self._fire_id,   # ← add these
    self._idx_buf,
)

# And update the @nb.njit function signature to match:
@nb.njit(cache=True, fastmath=True)
def simulate_step(grid, active, active_next,
                  metadata, moved,
                  dirty, dirty_next,
                  density_lut, move_lut, viscosity_lut,
                  steam_id, water_id, lava_id, obs_id,
                  oil_id, fire_id,          # ← add these
                  idx_buf):
```

Step 3 — Write the reaction as a `@nb.njit` function

Add it near the other reaction functions (`_react`, etc.):

```python
@nb.njit(cache=True, fastmath=True)
def _react_oil_fire(grid, metadata, active_next, dirty_next,
                    r, c, oil_id, fire_id, H, W, CH, CW):
    """Oil touching fire ignites: the oil cell becomes fire."""
    for di in range(4):
        if   di == 0: nr, nc = r - 1, c
        elif di == 1: nr, nc = r + 1, c
        elif di == 2: nr, nc = r,     c - 1
        else:         nr, nc = r,     c + 1

        if not (0 <= nr < H and 0 <= nc < W):
            continue
        if grid[nr, nc] != fire_id:
            continue

        # Convert this oil cell to fire
        grid[r, c]     = fire_id
        metadata[r, c] = 80 + np.random.randint(0, 80)
        active_next[r, c] = True
        _mark_chunk(dirty_next, r, c, CH, CW)
        return
```

Step 4 — Call the reaction inside `simulate_step`

In the main per-cell loop, add a check before the `move = move_lut[cell]` line:

```python
# --- Oil ignition reaction ---
if cell == oil_id:
    _react_oil_fire(grid, metadata, active_next, dirty_next,
                    r, c, oil_id, fire_id, H, W, CH, CW)
    cell = grid[r, c]   # cell may have changed to fire
    if cell == 0:
        continue
```

Step 5 — Delete the Numba cache and relaunch

Any time you change a `@nb.njit` function's **signature** (adding/removing parameters), Numba's hash won't catch it and you can get a type-mismatch crash. It's safest to clear the cache after signature changes:

```bash
python clear_cache.py
```

(See Section 5 below for this script.)

---

5. Cache cleanup script

Save this as `clear_cache.py` next to your sim file and run it whenever you want a clean slate:

```python
```

```python
"""
clear_cache.py
--------------
Deletes all Numba-generated .nbi and .nbc files from __pycache__.
Regular Python .pyc bytecode is left alone.

Usage:
    python clear_cache.py            # preview what would be deleted
    python clear_cache.py --delete   # actually delete
"""
import pathlib, sys

cache_dir = pathlib.Path(__pycache__)
numba_files = list(cache_dir.glob("*.nbi")) + list(cache_dir.glob("*.nbc"))

if not numba_files:
    print("No Numba cache files found.")
    sys.exit(0)

total = sum(f.stat().st_size for f in numba_files)
print(f"Found {len(numba_files)} Numba cache files  ({total / 1024:.1f} KB)")
for f in sorted(numba_files):
    print(f"  {f.name}")

if "--delete" not in sys.argv:
    print("\nDry run — pass --delete to remove them.")
else:
    for f in numba_files:
        f.unlink()
    print(f"\nDeleted {len(numba_files)} files.")
```

When to run it

Situation	Action needed
You changed code **inside** a `@nb.njit` function	Nothing — Numba detects automatically
You changed a `@nb.njit` function's **parameter list**	Run `clear_cache.py --delete`
You **deleted or renamed** a `@nb.njit` function	Run `clear_cache.py --delete` (orphan files linger otherwise)
Numba or Python version upgrade	Run `clear_cache.py --delete`

Situation	Action needed
Unexplained crash or wrong behaviour after edits	Run `clear_cache.py --delete` as a first debugging step

6. Quick-reference cheat sheet

```
Add element   → colour constant → Element(...) at end of list → keyboard mapping
Add reaction  → cache ID in __init__ → pass to simulate_step → @nb.njit func →
call in loop
Changed signature? → python clear_cache.py --delete  then relaunch
```